

Tarea #1

05.2026.IC.90501.G1.TECNICAS DE
PROGRAMACION

Stephanie Solis

Cristian Casares

Entregable #1

Proyecto #1 – Nutrición para todos

Introducción.....	4
Descripción del Proyecto.....	4
Objetivos del Proyecto.....	5
Objetivo Principal.....	5
Objetivos Secundarios.....	5
Objetivos del Sistema.....	5
Arquitectura del Sistema MVC.....	6
Backlog del Proyecto.....	7
EPIC 1 – Gestión de Usuarios.....	7
PBI 1.1.1 – Crear Usuario.....	7
User Story.....	7
Criterios de aceptación.....	7
Subtareas técnicas.....	7
PBI 1.1.2 – Editar Usuario.....	8
User Story.....	8
Criterios de aceptación.....	8
Subtareas técnicas.....	8
PBI 1.1.3 – Eliminar Usuario.....	8
User Story.....	8
Criterios de aceptación.....	8
Subtareas técnicas.....	9
EPIC 2 – Gestión de Productos.....	9
PBI 2.1.1 – Crear Producto.....	9
User Story.....	9
Criterios de aceptación.....	9
Subtareas técnicas.....	10
PBI 2.1.2 – Editar Producto.....	10
User Story.....	10
Criterios de aceptación.....	10
Subtareas técnicas.....	10
PBI 2.1.3 – Eliminar Producto.....	11
User Story.....	11
Criterios de aceptación.....	11
Subtareas técnicas.....	11
EPIC 3 – Gestión de Menús.....	12
PBI 3.1.1 – Crear Menú Diario.....	12
User Story.....	12
Criterios de aceptación.....	12

Subtareas técnicas.....	12
PBI 3.1.2 – Agregar Productos al Menú.....	12
User Story.....	12
Criterios de aceptación.....	12
Subtareas técnicas.....	13
EPIC 4 – Información Nutricional.....	13
PBI 4.1.1 – Calcular IMC.....	13
User Story.....	13
Criterios de aceptación.....	13
Subtareas técnicas.....	13
PBI 4.1.2 – Calcular Calorías y carbos.....	14
User Story.....	14
Criterios de aceptación.....	14
Subtareas técnicas.....	14
EPIC 5 – Estadísticas.....	15
PBI 5.1.1 – Consumo vs Meta.....	15
User Story.....	15
Criterios de aceptación.....	15
Subtareas técnicas.....	15
PBI 5.1.2 – Estadísticas por Rango.....	15
User Story.....	15
Criterios de aceptación.....	15
Subtareas técnicas.....	16
EPIC TÉCNICO – Infraestructura y Calidad.....	16
Objetivo.....	16
Propósito.....	16
Alcance.....	16
TEC 1 – Configurar estructura MVC.....	16
TEC 2 – Configurar proyecto WinForms.....	17
TEC 3 – Configurar persistencia base.....	17
TEC 4 – Manejo global de excepciones.....	18
TEC 5 – Refactorización.....	18
TEC 6 – Pruebas integrales.....	19
Sprints - Planificación y metodología.....	19
Riesgos del Proyecto.....	20
Conclusiones / Análisis de Aprendizaje.....	20

Introducción

El presente documento corresponde al Entregable #1 del Proyecto #1 – Nutrición para Todos, desarrollado en el curso Técnicas de Programación. En esta etapa se define la planificación integral del sistema mediante la estructuración del trabajo en Epics y Product Backlog Items (PBIs), estableciendo el alcance funcional y técnico del proyecto.

El sistema, denominado Dragon Nutrex, será una aplicación de escritorio desarrollada en C# utilizando Windows Forms, cuyo propósito es permitir la gestión de información nutricional de manera organizada y automatizada. La solución contempla la administración de usuarios, productos alimenticios, menús diarios y estadísticas, incorporando cálculos como IMC, calorías objetivo y distribución de macronutrientes.

Desde el enfoque técnico, el proyecto se basa en la arquitectura Modelo–Vista–Controlador (MVC), garantizando una adecuada separación de responsabilidades y facilitando la mantenibilidad, escalabilidad y claridad estructural del sistema. Asimismo, se define una base de infraestructura que incluye persistencia de datos, manejo de excepciones y aplicación de buenas prácticas de desarrollo (SOLID).

Este entregable establece los fundamentos necesarios para una implementación ordenada, reduciendo riesgos y asegurando coherencia entre los requerimientos funcionales y la arquitectura propuesta.

Descripción del Proyecto

Dragon Nutrex es una aplicación de escritorio desarrollada en C# con Windows Forms que permite la gestión integral de información nutricional. El sistema administra usuarios, productos alimenticios y menús diarios, incorporando cálculos automáticos de IMC, calorías objetivo y distribución de macronutrientes. Asimismo, genera estadísticas comparativas de consumo versus meta, apoyándose en una arquitectura

MVC que garantiza separación de responsabilidades, mantenibilidad y escalabilidad del sistema. Objetivo del Sistema

Objetivos del Proyecto

Objetivo Principal

Desarrollar una aplicación de escritorio en C# utilizando Windows Forms bajo arquitectura MVC, aplicando principios SOLID, prácticas de Clean Code y buenas prácticas de ingeniería de software, que permita la gestión integral de información nutricional mediante operaciones CRUD y garantice una estructura mantenible y escalable.

Objetivos Secundarios

- Diseñar e implementar la arquitectura MVC integrando principios SOLID, estándares de Clean Code y manejo adecuado de excepciones, asegurando separación de responsabilidades y calidad estructural del sistema.
- Desarrollar funcionalidades CRUD para la administración de usuarios, productos y menús, incorporando validaciones de datos y persistencia estructurada (CSV/JSON).
- Entregar documentación técnica y planificación estructurada del sistema (Epics, PBIs, criterios de aceptación y backlog), evidenciando la correcta aplicación de metodologías ágiles y buenas prácticas de desarrollo.

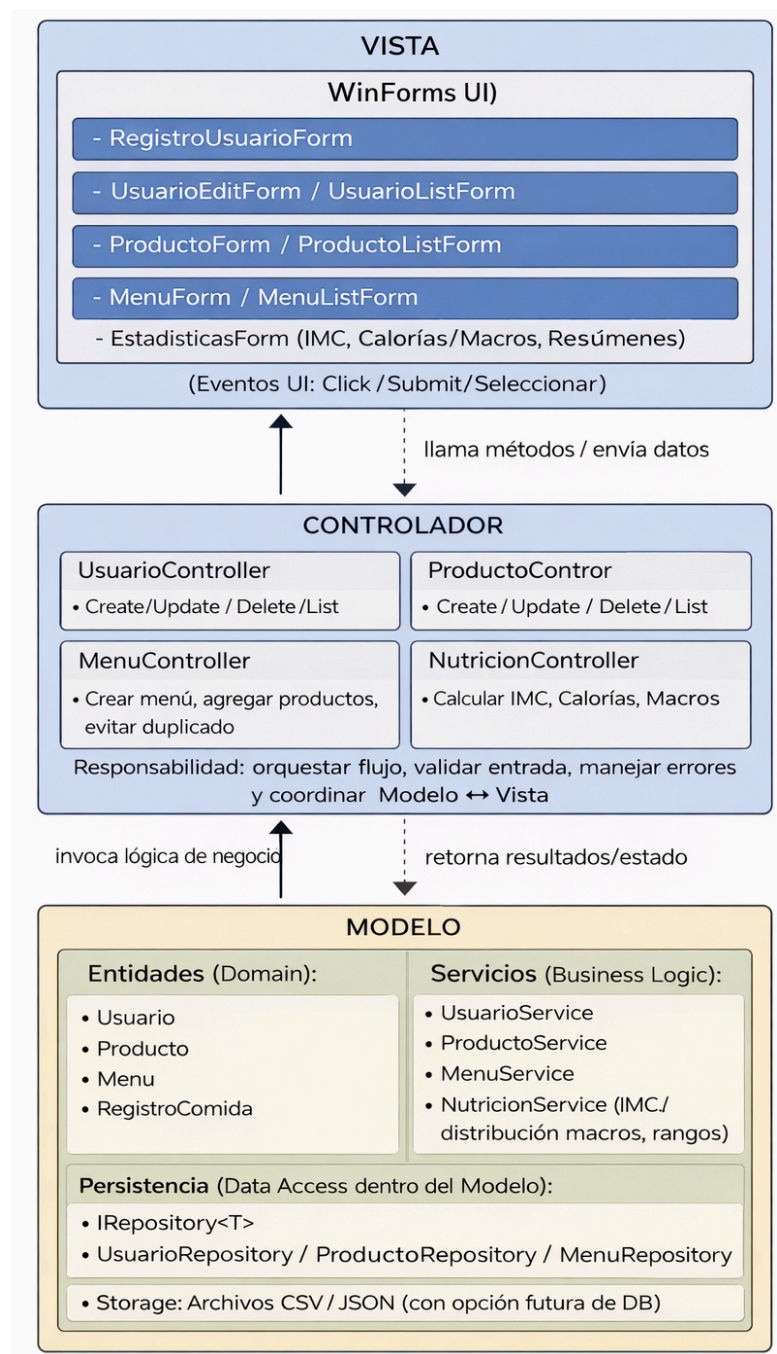
Objetivos del Sistema

Proveer una solución de software de escritorio que permita gestionar de manera estructurada la información nutricional de los usuarios, mediante la administración de perfiles, productos y menús diarios, incorporando cálculos automáticos y estadísticas comparativas, bajo una arquitectura MVC que garantice calidad, mantenibilidad y correcta aplicación de principios de ingeniería de software.

Arquitectura del Sistema MVC

Este diagrama representa la arquitectura MVC (Modelo-Vista-Controlador) de la aplicación, mostrando cómo la Vista (WinForms) gestiona la interacción del usuario, el Controlador coordina las operaciones y valida los datos, y el Modelo contiene las entidades, la lógica de negocio y la capa de persistencia.

También se visualiza el flujo de comunicación entre capas, asegurando una clara separación de responsabilidades y facilitando el mantenimiento y la escalabilidad del sistema.





Backlog del Proyecto

EPIC 1 – Gestión de Usuarios

PBI 1.1.1 – Crear Usuario

User Story

Como usuario del sistema, quiero registrar mis datos personales y nutricionales, para que el sistema pueda calcular mis metas y realizar seguimiento.

Criterios de aceptación

1. Permite ingresar peso, altura, actividad, objetivo y tipo de dieta.
2. Valida campos obligatorios.
3. No permite valores negativos.
4. Guarda la información en archivo.
5. Recupera los datos al reiniciar la aplicación.

Subtareas técnicas

- Crear clase Usuario (Model)
- Definir propiedades y validaciones básicas
- Crear formulario RegistroUsuarioForm
- Implementar UsuarioController
- Implementar UsuarioService
- Implementar UsuarioRepository
- Implementar método Create/GuardarUsuario()
- Persistencia en archivo
- Manejo de excepciones
- Pruebas manuales

PBI 1.1.2 – Editar Usuario

User Story

Como usuario del sistema, quiero modificar mis datos personales y nutricionales, para mantener mi información actualizada y que los cálculos reflejen cambios recientes.

Criterios de aceptación

1. Permite seleccionar un usuario existente.
2. Permite modificar peso, altura, actividad, objetivo y tipo de dieta.
3. Valida campos obligatorios.
4. No permite valores negativos.
5. Actualiza la información en el archivo.
6. Los cambios se mantienen al reiniciar la aplicación.

Subtareas técnicas

- Implementar método ActualizarUsuario()
- Crear pantalla de edición
- Cargar datos existentes en formulario
- Reutilizar validaciones existentes
- Actualizar persistencia
- Refrescar vista tras guardar
- Manejo de excepciones
- Pruebas manuales

PBI 1.1.3 – Eliminar Usuario

User Story

Como usuario del sistema, quiero eliminar un perfil de usuario, para mantener actualizada la información y evitar registros innecesarios.

Criterios de aceptación

1. Permite seleccionar un usuario existente para eliminar.

2. Solicita confirmación antes de eliminar el usuario.
3. Elimina correctamente el usuario del archivo de persistencia.
4. El usuario eliminado deja de aparecer en el sistema.
5. Maneja adecuadamente la eliminación de datos asociados (menús y registros).

Subtareas técnicas

- Implementar método EliminarUsuario() en UsuarioController
- Implementar método Delete() en UsuarioRepository
- Actualizar persistencia de datos
- Validar eliminación de datos asociados (menús/registros)
- Confirmación en la interfaz de usuario
- Refrescar listado tras eliminación
- Manejo de excepciones
- Pruebas funcionales

EPIC 2 – Gestión de Productos

PBI 2.1.1 – Crear Producto

User Story

Como usuario, quiero registrar productos con valores nutricionales, para utilizarlos en mis menús diarios.

Criterios de aceptación

1. Permite ingresar nombre y carbo.
2. Valida que carbo ≥ 0 .
3. Guarda el producto correctamente.
4. Aparece en el listado dinámico.

Subtareas técnicas

- Crear clase Producto
- Definir propiedades nutricionales
- Implementar validaciones (carbos ≥ 0)
- Crear formulario ProductoForm
- Implementar ProductoController
- Implementar ProductoService
- Implementar ProductoRepository
- Método GuardarProducto()
- Persistencia en archivo
- Pruebas funcionales

PBI 2.1.2 – Editar Producto

User Story

Como usuario, quiero modificar la información de un producto, para corregir o actualizar sus valores nutricionales.

Criterios de aceptación

1. Permite seleccionar un producto existente.
2. Permite modificar nombre y valores nutricionales.
3. Valida que carbos ≥ 0 .
4. Guarda correctamente los cambios.
5. Actualiza el listado dinámico.

Subtareas técnicas

- Implementar método ActualizarProducto()
- Cargar producto en formulario
- Validaciones de datos
- Actualizar persistencia de datos

- Refrescar listado en UI
- Manejo de excepciones
- Pruebas funcionales

PBI 2.1.3 – Eliminar Producto

User Story

Como usuario, quiero eliminar un producto, para mantener actualizado el catálogo de alimentos.

Criterios de aceptación

1. Permite seleccionar un producto para eliminar.
2. Solicita confirmación antes de eliminar.
3. El producto deja de aparecer en el listado.
4. Se actualiza la persistencia de datos.

Subtareas técnicas

- Implementar método EliminarProducto()
- Confirmación en UI
- Actualizar persistencia
- Refrescar listado
- Manejo de excepciones
- Pruebas manuales

EPIC 3 – Gestión de Menús

PBI 3.1.1 – Crear Menú Diario

User Story

Como usuario, quiero crear un menú por fecha, para registrar mis comidas diarias.

Criterios de aceptación

1. Permite seleccionar fecha.
2. Evita duplicados por fecha.
3. Guarda correctamente el menú.

Subtareas técnicas

- Crear clase Menu
- Relacionar Usuario con Menu
- Validar duplicado por fecha
- Crear formulario de selección
- Implementar MenuController
- Implementar persistencia
- Pruebas manuales

PBI 3.1.2 – Agregar Productos al Menú

User Story

Como usuario, quiero agregar productos a un menú diario, para registrar mis comidas y calcular mis totales nutricionales.

Criterios de aceptación

1. Permite seleccionar producto y cantidad.
2. Valida que cantidad > 0.
3. Calcula automáticamente calorías.



4. Calcula automáticamente macronutrientes.
5. Actualiza totales en tiempo real.
6. Guarda el registro correctamente.

Subtareas técnicas

- Crear clase RegistroComida
- Implementar relación Menu-Producto
- Implementar cálculo automático de calorías
- Implementar cálculo automático de macronutrientes
- Validar cantidad > 0
- Actualizar totales en tiempo real
- Persistencia de registros
- Refrescar vista
- Pruebas funcionales

EPIC 4 – Información Nutricional

PBI 4.1.1 – Calcular IMC

User Story

Como usuario, quiero visualizar mi IMC, para conocer un indicador básico de salud.

Criterios de aceptación

1. Calcula IMC correctamente.
2. Maneja división por cero.
3. Muestra resultado en pantalla.

Subtareas técnicas

- Implementar método CalcularIMC()
- Validar altura > 0



- Mostrar resultado en UI
- Pruebas de cálculo

PBI 4.1.2 – Calcular Calorías y carbos

User Story

Como usuario, quiero conocer mis calorías recomendadas y distribución de macronutrientes, para comparar con mi consumo diario.

Criterios de aceptación

1. Calcula calorías recomendadas según objetivo.
2. Calcula distribución de carbos según tipo de dieta.
3. Muestra resultados en pantalla.
4. Se actualiza automáticamente si cambian los datos del usuario.

Subtareas técnicas

- Implementar método Calcular Calorias Objetivo()
- Implementar método CalcularDistribucioncarbos()
- Aplicar reglas según tipo de dieta
- Mostrar resultados en UI
- Actualización automática si cambia perfil
- Pruebas de cálculo

EPIC 5 – Estadísticas

PBI 5.1.1 – Consumo vs Meta

User Story

Como usuario, quiero comparar mi consumo diario con mi meta, para monitorear mi progreso.

Criterios de aceptación

1. Muestra calorías consumidas.
2. Muestra diferencia respecto a la meta.
3. Muestra carbohidratos consumidos.

Subtareas técnicas

- Implementar método ObtenerResumenDiario()
- Calcular diferencia vs meta
- Mostrar resultados en UI
- Manejar caso sin registros
- Pruebas funcionales

PBI 5.1.2 – Estadísticas por Rango

User Story

Como usuario, quiero visualizar estadísticas en un rango de fechas, para analizar mi progreso nutricional.

Criterios de aceptación

1. Permite seleccionar fecha inicio y fecha fin.
2. Valida que fecha inicio $\leq$ fecha fin.
3. Calcula totales en el rango seleccionado.
4. Calcula promedios de consumo.

5. Muestra resultados en pantalla.

Subtareas técnicas

- Implementar método ObtenerResumenPorRango()
- Validar fecha inicio $\leq$ fecha fin
- Calcular totales
- Calcular promedios
- Mostrar resultados en pantalla
- Pruebas funcionales

EPIC TÉCNICO – Infraestructura y Calidad

Objetivo

Establecer la base técnica del sistema Dragon Nutrex asegurando una arquitectura organizada, mantenible y alineada a buenas prácticas de desarrollo.

Propósito

Garantizar que el sistema tenga una estructura sólida que permita escalabilidad, control de errores y correcta separación de responsabilidades.

Alcance

- Configuración de arquitectura MVC
- Implementación de persistencia por módulo
- Manejo global de excepciones
- Aplicación de principios SOLID
- Pruebas integrales del sistema

TEC 1 – Configurar estructura MVC

- Crear solución/proyecto y estructura de carpetas:
Models/Views/Controllers/Services/Repositories/Utils

- Definir convención de nombres (Forms, Controllers, Services, Repositories)
- Crear clases base del dominio vacías: Usuario, Producto, Menu, RegistroComida
- Crear Controllers base (stubs): UsuarioController, ProductoController, MenuController, NutricionController
- Crear Services base (stubs): UsuarioService, ProductoService, MenuService, NutricionService
- Crear Repositories base (stubs): UsuarioRepository, ProductoRepository, MenuRepository
- Validar separación de responsabilidades (no lógica en Forms, lógica en Services)

TEC 2 – Configurar proyecto WinForms

- Crear proyecto WinForms y validar ejecución
- Configurar Program.cs (entry point)
- Crear MainForm (menú principal / navegación)
- Definir navegación básica (botones/menú a: Usuarios, Productos, Menús, Nutrición, Estadísticas)
- Crear Forms placeholders (vacíos) para cada módulo:
 - RegistroUsuarioForm, UsuarioListForm
 - ProductoForm, ProductoListForm
 - MenuForm, MenuListForm
 - NutricionForm (IMC/carbos)
 - EstadisticasForm
- Prueba: abrir/cerrar formularios sin errores

TEC 3 – Configurar persistencia base

- Definir interfaz genérica IRepository<T> (Create/Update/Delete/GetAll/GetById)
- Definir modelo de almacenamiento (JSON o CSV) y estructura de carpetas /Data
- Implementar clase helper FileStorage (leer/escribir archivo)
- Implementar repositorio base genérico (si aplica) o repos base por entidad



- Implementar persistencia inicial para Usuarios (para validar end-to-end)
- Implementar métodos de serialización/deserialización
- Manejo básico de errores de archivo (archivo no existe, vacío, corrupto)
- Prueba: guardar y cargar datos tras reiniciar app

TEC 4 – Manejo global de excepciones

- Configurar handler global (AppDomain/ThreadException para WinForms)
- Crear clase Logger (guardar logs en archivo)
- Definir formato de log (fecha, módulo, excepción, mensaje)
- Implementar mensajes amigables al usuario (MessageBox)
- Centralizar manejo de validaciones (excepciones controladas vs no controladas)
- Probar escenarios:
 - Archivo no existe
 - Error de parseo JSON/CSV
 - Valores inválidos (negativos, cero, etc.)
- Verificar que la app no se cae ante errores comunes

TEC 5 – Refactorización

- Identificar y eliminar duplicación (validaciones repetidas, lectura/escritura repetida)
- Extraer utilidades comunes (validaciones, conversión, helpers)
- Revisar responsabilidades (Forms sin lógica, funcionamiento de Controllers, Services calculan, Repos guardan)
- Aplicar principios SOLID
- Renombrar clases/métodos para consistencia
- Limpiar warnings / code smells
- Revisión final del flujo MVC (dependencias correctas)

TEC 6 – Pruebas integrales

- Definir checklist de pruebas por flujo completo:
 - Crear/Editar/Eliminar Usuario

- Crear/Editar/Eliminar Producto
- Crear Menú por fecha (sin duplicados)
- Agregar productos al menú (cantidad > 0, cálculos)
- Calcular IMC / calorías / carbo
- Estadísticas: consumo vs meta y por rango
- Probar persistencia completa (cerrar app y reabrir)
- Probar casos borde:
 - Sin usuarios / sin productos / sin menús
 - Fechas inválidas (inicio > fin)
 - Cantidad 0 o negativa
- Registrar evidencias (capturas o documento breve de resultados)
- Corrección de bugs encontrados + rerun de pruebas
- Validación final para entrega/demo

Sprints - Planificación y metodología

El proyecto Dragon Nutrex se desarrollará en cuatro sprints organizados de manera incremental, permitiendo construir el sistema por módulos funcionales y asegurar avances progresivos y controlados. Cada sprint tendrá objetivos definidos y entregables específicos alineados con la arquitectura MVC.

La gestión del trabajo se realizará en JIRA bajo la metodología Scrum, utilizando Epics, PBIs y subtareas para planificar, priorizar y dar seguimiento al desarrollo, garantizando organización, trazabilidad y control del progreso del proyecto.

Sprint	Fechas	Contenido (Epics y PBIs)
Sprint 1	8 - 13 marzo	EPIC Técnico (TEC 1, TEC 2, TEC 3) + EPIC 1 (PBI 1.1.1, 1.1.2, 1.1.3)
Sprint 2	14 - 20 marzo	EPIC 2 – Gestión de Productos (PBI 2.1.1, 2.1.2, 2.1.3)
Sprint 3	21 - 27 marzo	EPIC 3 – Gestión de Menús (PBI 3.1.1, 3.1.2) + EPIC 4 – Información Nutricional (PBI 4.1.1, 4.1.2)
Sprint 4	28 - 30 marzo	EPIC 5 – Estadísticas (PBI 5.1.1, 5.1.2) + EPIC Técnico (TEC 4, TEC 5, TEC 6)

Riesgos del Proyecto

- Errores en validaciones.
- Problemas en persistencia de datos.
- Tiempo limitado para pruebas.

Conclusiones / Análisis de Aprendizaje

El desarrollo del Entregable #1 permitió aplicar de forma práctica la planificación ágil mediante Epics y PBIs en Jira, facilitando la organización del trabajo y la correcta división entre funcionalidades y tareas técnicas. La estructuración del backlog evidenció la importancia de definir criterios de aceptación claros para reducir ambigüedades y riesgos desde etapas tempranas.

El análisis previo al desarrollo permitió comprender la importancia del uso de herramientas como Git, GitHub y Visual Studio dentro del proceso de ejecución del proyecto. En esta fase de pre-ejecución se identificó cómo el control de versiones facilitará la trazabilidad de cambios y la organización del trabajo durante la implementación. Asimismo, la definición anticipada de una arquitectura basada en MVC permitió planificar una correcta separación de responsabilidades y bajo acoplamiento, estableciendo una base técnica sólida antes de iniciar la programación.

En conjunto, el proceso demostró que una planificación estructurada, apoyada en herramientas profesionales, mejora la calidad del diseño y facilita el desarrollo ordenado y mantenible del sistema.